



Code Hardening: C/C++

CEN429 Secure Programming — Week 4

Asst. Prof. Dr. Uğur CORUH · 09.10.2026

Today's Plan (3 Hours)

Hour	Section	Topic
1	1	Layers of hardening · SEI CERT · input validation principles · CERT pairs
2	2–4	Format string Demo 1 · UAF Demo 2 · integer/UB Demo 3 · error handling/signals · static analysis · sanitizers · fuzzing Demo 4
3	5–7	Compiler/OS protections Demo 5 · secure build pipeline (CI) · code obfuscation Demo 6–7 · end to end · project

Learning outcome: LO.3 (binary application protections)

A Brief History — the Race Between Memory Bugs and Defences

- 1988 — **Morris Worm** announces the buffer overflow to the world
- 1996 — *Smashing the Stack for Fun and Profit* teaches exploitation to everyone
- 1998 → 2004 — **canary** (StackGuard) · **ASLR** (PaX) · **DEP/NX** · FORTIFY
- 2012–13 — **AddressSanitizer** and **fuzzing** (AFL) catch bugs automatically

This course follows this race: write the bug → let the tool catch it → let the compiler/OS protect.

What We Bring from Earlier Weeks

- **Process memory: stack and heap** — the stack automatically manages the local variables of function calls, the heap is managed manually with `malloc` / `free` (**Week 1**)
- **Buffer overflow** — writing more data to an array than its allocated size; the overflowing bytes land on neighbouring memory (**Week 1**)
- **Stack frame and return address** — the region a function call gets on the stack, holding its local variables and the return address (**Week 1**)
- **The white-box attacker model** — the owner of the device is also a possible attacker; they can read memory, attach a debugger (**Week 1**)
- **CWE** — a numbered catalogue of software weaknesses (**Week 2**)

This week: on top of these foundations we build SEI CERT rules, sanitizers, and compiler/OS protections — applying the Weeks 1–3 principles/threat-model/crypto foundation to the C/C++ code itself.

This Week's Concepts

Each term is defined once, where it first appears in the body; here we only mark **where**.

Concept	Where
Layers of hardening · SEI CERT	Section 1
Input validation principles	Section 1
Format string vulnerability	Section 2
Use-after-free (UAF)	Section 3
Undefined behaviour (UB)	Section 3
Error handling, signals	Section 3
Static analysis · sanitizers · fuzzing	Section 4
Compiler/OS protections · CI	Section 5
Code obfuscation	Section 6

Pointer

- **Pointer:** a variable that holds a memory **address**.
- `p` is an address; `*p` is the value at that address.
- Wrong address → crash or wrong data.

Compiler Flag

- **Flag:** an option passed to the compiler.
- Example: `-O2` (optimisation), `-Wall` (warnings), `-fsanitize=address`.
- The right flags catch many bugs **at compile time**.

Warning vs Error

- **Error:** compilation stops.
- **Warning:** compilation continues but a problem is reported.
- Rule: **turn warnings into errors** (`-Werror`) — an ignored warning is a future vulnerability.

How Do the Demos Work?

- Demos live under `code/week-04` ; they run on **Windows (Visual Studio 2022)** and **WSL/Linux**.
- Build: Windows `.\build.ps1` · WSL/Linux `./build.sh`
- Run: in each demo folder, Windows `.\demo.ps1` · WSL/Linux `sh demo.sh`
- **Fuzzing (Demo 4)** needs **clang** on Linux/WSL (`sudo apt install -y clang`); on Windows it needs Visual Studio's "C++ AddressSanitizer" component.

 **Ethics:** the demos run only against **our own small programs**; none of them touch another program or system.



1. Layers and SEI CERT

Three Layers, the Right Order

Code hardening is three layers. The order matters:

1. **First, bug-free code** (secure coding)
2. **Then, limit the damage** (compiler/OS protections)
3. **Finally, make it harder to read** (obfuscation)



Obfuscation does **not fix** the bug.

Why This Order?

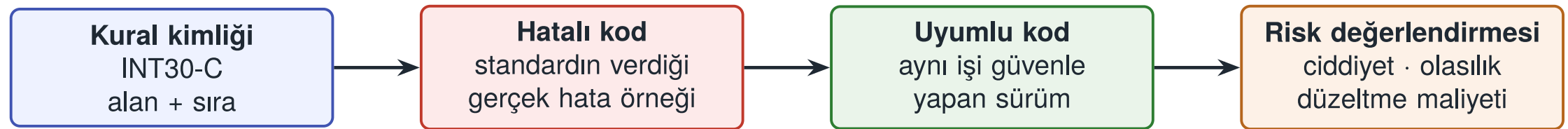
- Adding protection to buggy code is like painting over a rotten foundation.
- First **remove** the bug; then **catch** what slips through; finally **slow down** reverse engineering.
- We will see all three this week, in this order.

Anatomy of a CERT Rule — Diagram

SEI CERT C/C++: the rulebook of secure coding, distilled from real-world vulnerabilities.

SEI CERT kuralının anatomisi

her kural aynı dört parçayla gelir

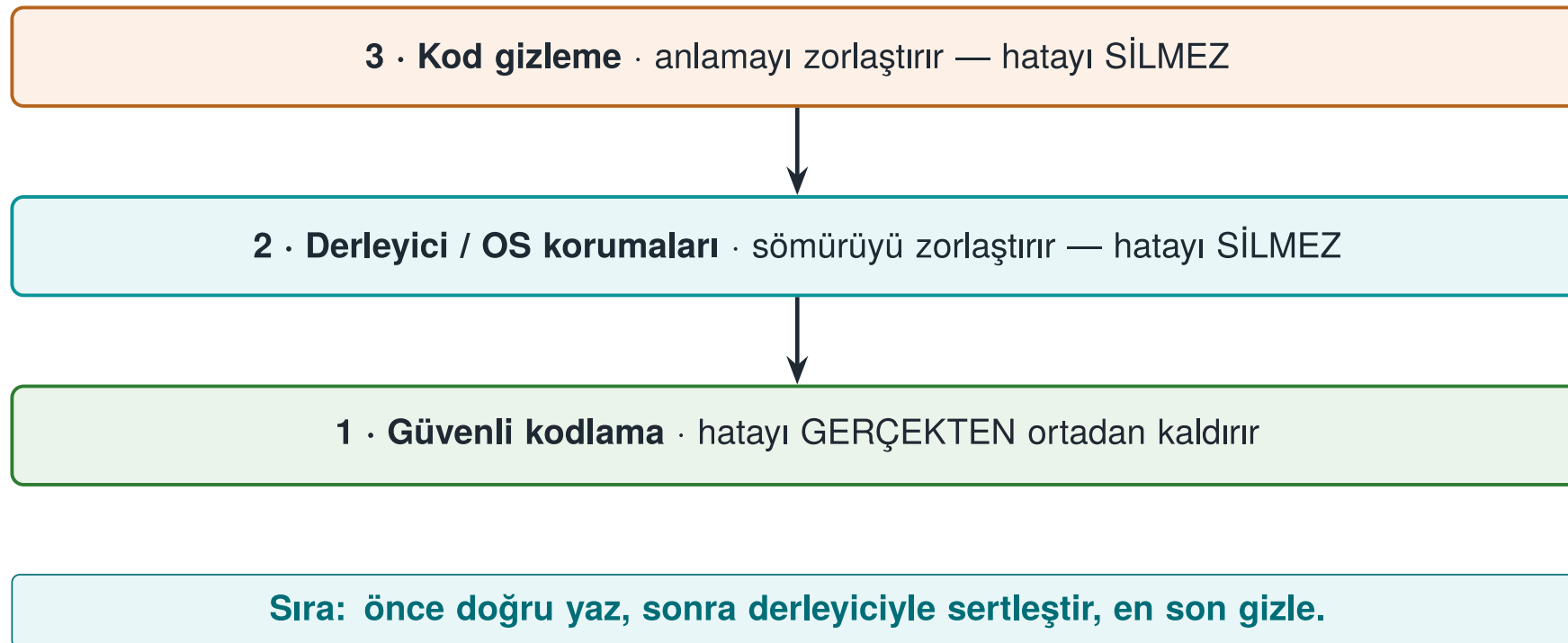


Kuralı ezberlemeyin: hatalı ile uyumlu kodun FARKINI okuyun; öğretici olan orasıdır.

Three Layers — Diagram

Kod sağlamlaştırmanın üç katmanı

yalnız en alttaki katman hatayı yok eder



Layer 2 · Secure Coding

- **Question:** is there a bug in my code?
- **Tools:** CERT rules, sanitizers, fuzzing.
- **Goal:** not writing the bug in the first place.

Layer 3 · Compiler/OS

- **Question:** if a bug slips through, is it hard to exploit?
- **Tools:** canary, FORTIFY, ASLR, NX, RELRO, CFI.
- **Goal:** making exploitation of the remaining bug expensive.

Layer 4 · Obfuscation

- **Question:** does someone reading the code understand it?
- **Tools:** symbol/string hiding, removing logging, flattening.
- **Goal:** slowing down reverse engineering.

From the Field: Scheme Requirements

In a certified product:

- "Common vulnerabilities must be addressed: sensitive data, injection, overflow, language-specific weaknesses"
- "Defensive programming **consistent across the entire codebase**"
- "Protection against static and dynamic reverse engineering"

➔ Eighteen hardening measures on the native side **alone**.

SEI CERT: What Does a Rule Look Like?

Parts of every CERT rule:

- **Identifier:** STR31-C , MEM50-CPP
- **Title:** one sentence
- **Noncompliant example** and **compliant solution**
- **Risk level** (L1/L2/L3) and a **CWE** link

Rule or Recommendation?

- **Rule:** violating it is a **vulnerability**, and it can be checked automatically.
- **Recommendation:** good practice.
- Start a review with **L1** (highest risk) rules.

How Is the Risk Level Determined?

Risk = **severity** × **likelihood** × **cost to fix**.

- **L1**: high risk, these first.
- **L2 / L3**: progressively lower.

CERT Categories (1)

Abbreviation	Category	Example
INT	Integers	INT32-C: do not allow signed overflow
ARR/STR	Array/string	STR31-C: allocate sufficient space
MEM	Memory	MEM30-C: do not access freed memory
FIO	File I/O	FIO30-C: exclude input from the format string

CERT Categories (2)

Abbreviation	Category	Example
ENV	Environment	ENV33-C: do not call <code>system()</code>
SIG	Signals	SIG30-C: only safe fn. in the handler
ERR	Error handling	ERR33-C: detect a library error
CON/MSC	Concurrency/misc	CON43-C, MSC32-C

C++: MEM50-CPP, CTR50-CPP, STR50-CPP, EXP53-CPP, ERR50-CPP.

Automated CERT Auditing

- Warnings: `-Wall -Wextra -Wformat=2 -Wconversion`
- `clang-tidy cert-*`
- `cppcheck --addon=cert`
- MSVC `/analyze`

The evaluator runs this scan first, then reads by hand.

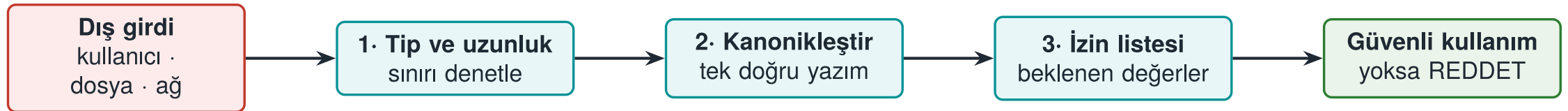


Input Validation

Input Validation — Diagram

Güven sınırında girdi doğrulama

her dış girdi bu kapıdan geçer



Kara liste değil İZİN listesi: neyin yasak olduğunu değil, neyin serbest olduğunu yazın.

Where Does Input Come From?

From more places than you'd think. And **all of it is untrusted**:

- Command line, environment variables
- File, network
- IPC (another process on the same machine)
- Database, library interface (JNI)

Commonly Forgotten Inputs

- Even `argv[0]` can be the attacker's; `PATH`, `LANG`.
- **Length fields** inside a file format.
- Network: says "16 bytes," sends 1 byte.
- "Our own data" may have changed since.

Five Principles of Input Validation

1. **Allow-list** (enumerate what's permitted), not a deny-list.
2. **Canonicalize first**, then validate.
3. **Length** → **type** → **range** → **format**, in that order.
4. At the **trust boundary**, as early as possible.
5. **Reject**, don't "clean up" and use it anyway.

Principle 1 · Allow-List

- **Deny-list:** "these are forbidden" — something always gets forgotten.
- **Allow-list:** "only these are permitted" — a safe default.

Example: a username may only contain `[a-z0-9_]`.

Principle 2 · Canonicalize

- The same thing, spelled differently: `/tmp/../../etc` , `%2e%2e/` .
- Reduce it to **one form** (canonicalize) first, then check it.
- Otherwise the check can be bypassed.

Principle 3 · strtol, Not atoi

```
errno = 0;
long value = strtol(s, &end, 10);
if (end == s) return -1; /* no digits at all */
if (*end != '\0') return -1; /* "12abc" */
if (errno == ERANGE) return -1; /* overflowed */
if (value < min_val || value > max_val) return -1; /* business rule */
```

Why Is `atoi` Dangerous?

- `atoi("abc")` = 0 (the error is silent)
- `atoi("999999999999")` = **undefined** (overflow)
- `atoi("12abc")` = 12 (the extra characters are ignored)

`strtol` reports **every error** to you; `atoi` does not.

Principle 3 · a Length-Prefixed Record

```
if (remaining < 3) return -1;           /* not even a header */
uint16_t length = buffer[1] << 8 | buffer[2];
if (length > remaining - 3) return -1;  /* SOURCE: declared > received */
if (length > sizeof rec->data) return -1; /* DEST: doesn't fit */
memcpy(rec->data, buffer + 3, length);
```

Two Separate Checks, the Right Order

- **Source check:** is the declared length greater than the data that arrived?
- **Destination check:** does it fit in the destination buffer?
- Order: `remaining < 3` **first**, so `remaining - 3` does not wrap.
- In the field: every array coming from JNI is **re-validated** on the native side.

CERT Pair · STR31-C

```
strcpy(copy, name);                                /* WRONG */  
int n = snprintf(copy, sizeof copy, "%s", name); /* COMPLIANT */  
if (n < 0 || (size_t)n >= sizeof copy) { /* truncated */ }
```

`strcpy` does not know bounds; `snprintf` knows the size and reports **truncation**.

CERT Pair · INT30-C

```
size_t remaining = total - read_count; /* WRONG: read_count > total → huge number */  
if (read_count > total) return ERROR; /* COMPLIANT: check order first */  
size_t remaining = total - read_count;
```

Unsigned subtraction **wraps**; check the logic first.

CERT Pair · MEM30-C

```
for (n = head; n; n = n->next) free(n);           /* WRONG */  
while (n) { Node *s = n->next; free(n); n = s; }  /* COMPLIANT */
```

Reading `n->next` after `free(n)` = accessing freed memory.

CERT Pair · ERR33-C

```
FILE *f = fopen(path,"rb"); fread(t,1,n,f);          /* WRONG */  
if (!f) return ERROR;  
if (fread(t,1,n,f) < n && ferror(f)) return ERROR; /* COMPLIANT */
```

Check **every** return value; `fopen` can return NULL.

Rule: Tie the Finding to an Identifier

Match every finding to a CERT identifier:

"This line is a MEM30-C violation (CWE-416)."

This makes the finding **objective** and **searchable**.

Section 1 — Quick Check

1. The three layers, and the right order?
2. Difference between a rule and a recommendation?
3. Why is `atoi` more dangerous than `strtol` ?

Section 1 — Answers

1. **Order:** secure code (source) → **compiler/tool** protections → **OS/runtime** protections. First write it correctly, then harden it, then defend it at runtime.
2. A **rule** always applies, and violating it is a defect; a **recommendation** is good practice depending on context.
3. `atoi` does **not report** errors (undefined/0 on overflow/invalid input); `strtol` lets you check **both the error and the bound** via `endptr + ERANGE`.



2. Format String Vulnerability

Format String Vulnerability — Diagram

Biçim dizisi açığı

tek fark: biçim dizgesini kim belirliyor

printf(girdi)

girdi BİÇİM DİZGESİ sayılır

%x → yığını okur

%n → belleğe YAZAR

→ bilgi sızıntısı, kod akışı

printf("%s", girdi)

biçim dizgesi SABİT

girdi yalnız VERİ

güvenli

En tehlikelisi %n: yazdığı bayt sayısını belleğe yazar → seçili adrese yazma.

The Problem · `printf(input)`

```
printf(user_input);      /* WRONG */  
printf("%s", user_input); /* CORRECT */
```

In the first case, user input is interpreted as a **format string**.

Why Is It Dangerous?

`printf` treats the `%` markers in the format string as a **command**.

If the user supplies `%x`, `%s`, `%n`, they make the program do work.

Brief History · Format String Bugs

- **1999–2000** — real vulnerabilities in widely used FTP servers like **wu-ftpd** brought this bug class to wide attention.
- Attackers used user data that leaked into logging calls as a format string to leak information, even achieve remote code execution.
- Lesson: a "harmless-looking" marker like `%n` is actually a **write primitive**.
- Response: `-Wformat-security` at compile time, `_FORTIFY_SOURCE` at run time.

Marker · `%x` , `%p`

- **Reads** values off the stack and prints them.
- → **information leak** (addresses, secret values).
- Weakens ASLR (if an address leaks).

Marker · %s

- Treats a value on the stack as a **pointer** and reads the string at that address.
- Random address → crash or memory read.

Marker · `%n`

- **Writes** the number of characters printed so far to an address.
- → **writes to memory** → corruption, hijacking control.
- The most dangerous one.

The Same Bug Elsewhere

Not just `printf`:

`syslog`, `fprintf`, `snprintf`, `err` / `warn` — they all take a format string.

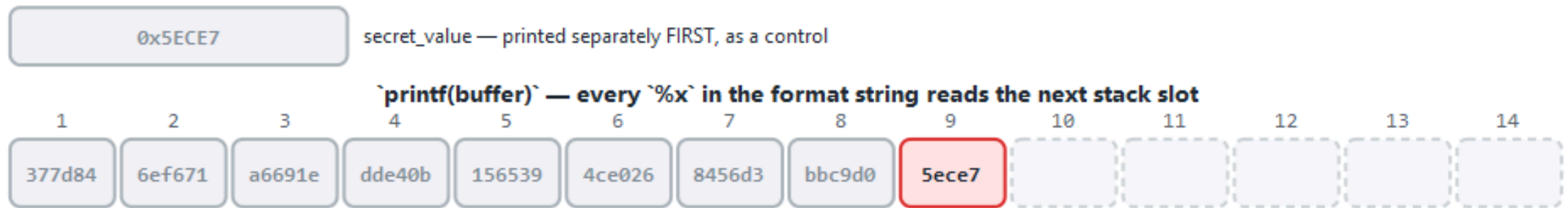
Rule: the format string is **always constant** (FIO30-C).

Demo 1 — Format String

code/week-04/01-format-string · CWE-134

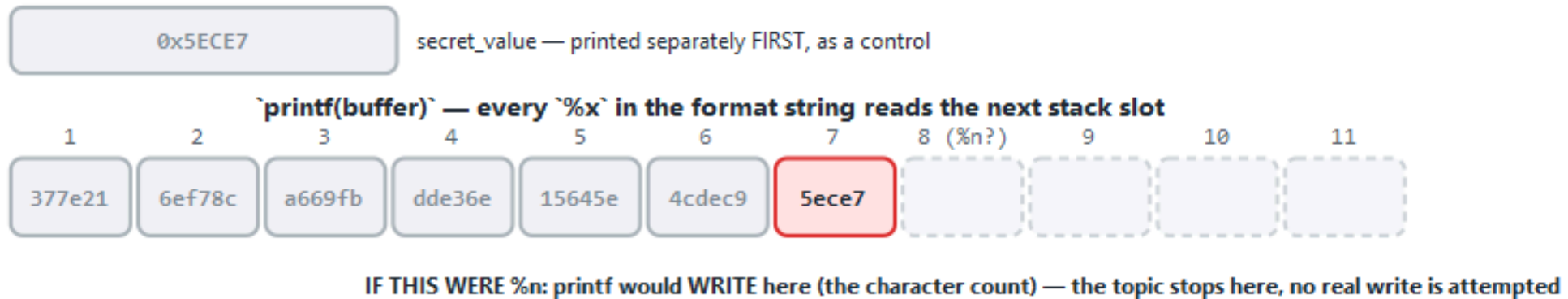
Step	What Is Seen?
0	GCC <code>-Wformat-security</code> warns; MSVC <code>/analyze</code> catches it
2 <code>%x.%x...</code>	The stack is dumped, the secret (synthetic) value appears
3 <code>AAAA%n</code>	Unprotected Linux crashes; Windows CRT rejects <code>%n</code>
4	<code>_FORTIFY_SOURCE : %n</code> in writable segment
5	Markers appear only as text

Animation — %x scans the stack



Result: the secret value was dumped to the screen. The one-line fix (`printf("%s", ...)`) makes this scan completely impossible.

Animation — Edge Case: What `%n` Would Do Next



Result: the secret value was dumped to the screen. The one-line fix (`printf("%s", ...)`) makes this scan completely impossible.

The Fix · Layer by Layer

1. The format string is always **constant** (FIO30-C).
2. Warning: `-Wformat -Wformat-security -Werror=format-security`.
3. Have the compiler **check** your own function too:

```
void gunluk_yaz(int duzey, const char *bicim, ...)  
    __attribute__((format(printf, 2, 3)));
```

4. Runtime: `_FORTIFY_SOURCE=2` ; MS CRT has `%n` disabled.



3. Memory: Use-After-Free

Stack and Heap — Diagram

Yığın (stack) ve öbek (heap)

taşmanın sonucu, belleğin hangi bölgesinde olduğuna göre değişir

YIĞIN

fonksiyonla açılır/kapanır
otomatik, hızlı
boyut derleme anında belli

taşma → dönüş adresi tehlikede

ÖBEK

malloc/free ile yönetilir
yaşam süresi size bağlı
boyut çalışma anında

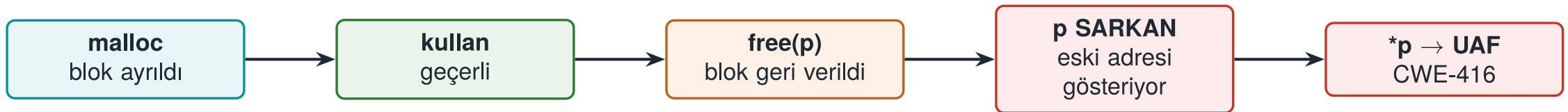
taşma → komşu blok / meta veri

Kanarya yığını korur; öbek taşmasını yakalamaz — bu yüzden ASan gerekir.

Use-After-Free — Diagram

Use-after-free zinciri

ASan raporundaki üç iz bu zincirin üç noktasıdır



Düzeltilme free ile use ARASINDADIR: p = NULL ve tek sahiplik kuralı.

What Is UAF? · Step 1

- An object is on the heap.
- **Two pointers** point to the same object.

UAF · Step 2

- One of them `free`s it.
- The other still holds that address → a **dangling** pointer.

UAF · Step 3

- The allocator gives the same block to **another object**.
- The dangling pointer now reads/writes **foreign data**.
- If the object has a **function pointer** (C++ virtual table), the risk grows.

Why Is UAF So Common?

Most of the "memory corruption fixed" notes in browser/OS updates fall into this class.

Complex ownership → a dangling pointer.

Demo 2 — UAF and Double Free

code/week-04/02-use-after-free · CWE-416/415 · MEM30-C

Step	What Is Seen?
1	The freed block gets reused → role <code>user</code> → <code>admin</code>
2 ASan	heap-use-after-free + locations
3	Double <code>free</code> → undefined behaviour
4 ASan	attempting double-free
5	<code>free</code> + <code>NULL</code> , single owner → safe

Animation — a small heap

A small heap: alloc() hands back the most-recently-freed chunk first (LIFO)



free list (LIFO), top = next

[A0, A1, A2]

Result: admin_panel was called through the dangling pointer — both the role and the behaviour became the attacker's data.

Animation — Edge Case: Double Free

A small heap: alloc() hands back the most-recently-freed chunk first (LIFO)



SECOND free — the same block is on the free list TWICE! The list is corrupted.

free list (LIFO), top = next

[A0, A1, A2, A3, A4, A4]

Result: a block freed twice can make the allocator hand the SAME memory to two different owners on a later malloc call — undefined behavior.

ASan Report · Three Stack Traces

```
ERROR: heap-use-after-free
#0 mode_uaf uaf.c:49  <- 1. USE
freed here:
#1 mode_uaf  uaf.c:38  <- 2. FREE
allocated here:
#1 mode_uaf      uaf.c:32  <- 3. ALLOCATION
```

The fix is usually at **location 2**: the ownership decision was wrong there.

Fix · the C Side

- Set the pointer to `NULL` after `free`.
- Establish a **single owner**, a single release point.
- Write down the ownership rule: "who releases this memory?"

Fix · C++ Smart Pointers

Need	C++
Single owner	<code>std::unique_ptr</code>
Shared	<code>std::shared_ptr</code>
Non-owning observation	<code>std::weak_ptr</code> → <code>lock()</code>
Array	<code>std::vector</code> , <code>std::array</code>

```
if (auto o = cache.lock()) use(*o); // otherwise stale memory is never accessed
```

⚠ `get()` is a raw pointer; when a `vector` grows, its iterators are invalidated.

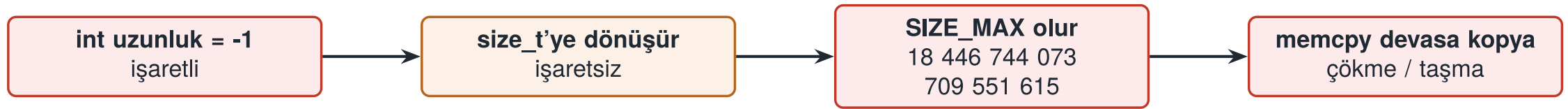


Integers and UB

Integer Overflow — Diagram

-1 ne zaman dev olur?

işaretili → işaretsiz dönüşümü sessizce gerçekleşir



Kural: uzunlukları size_t ile tutun; dışarıdan gelen sayıyı hem alt hem üst sınırla denetleyin.

Four Integer Bugs

Bug	Example	Result
Unsigned wraparound	<code>0u - 1</code> = 4,294,967,295	Logic error
Signed overflow	<code>INT_MAX + 1</code>	Undefined
Conversion loss	<code>long long</code> → <code>int</code>	Truncation
Shift	<code>1 << 31</code>	Undefined

Why Does It Matter? · Multiplication Overflow

```
uint32_t size = count * 4;    /* count = 0x40000001 → size = 4 */  
array = malloc(size);        /* small block */  
/* loop writes `count` times → heap overflow */
```

Writing too much into a small allocated block = an overflow.

UB · the Compiler Can Delete the Check

Undefined behaviour (UB): an operation whose result the C/C++ standard does not define (e.g., signed integer overflow).

```
if (x + 100 < x) /* assumes "if it overflows, it gets smaller" */  
    return -1; /* signed overflow is UB → the compiler MAY DELETE this branch */
```

- It "works" at `-00`, at `-02` the check **disappears**.
- UB cannot be relied on.

Correct: Check First

```
if ((b > 0 && a > INT_MAX - b) ||  
    (b < 0 && a < INT_MIN - b)) return false; /* before the overflow */  
if (__builtin_mul_overflow(a, b, &result)) return false; /* GCC/Clang */  
/* C23: <stdckdint.h> ckd_add/ckd_mul · MSVC: <intsafe.h> */
```

Catch the overflow **before it happens**.

Demo 3 — UB and UBSan

code/week-04/03-undefined-behavior · CWE-190/758

- Signed overflow, invalid shift, misaligned access
- On x86, most produce a wrong result **silently**
- `-fsanitize=undefined` → `signed integer overflow: 2147483647 + 1 ...`
- Windows: no UBSan → `/RTC`, `/analyze`, CERT

 `-fwrapv` **hides** the logic error; do not use it for diagnosis.

Animation — INT_MAX + add

`x = INT_MAX`; for each `add`, is it a bare `x + add`, or `checked_add(x, add, &y)`?

INT_MAX = 2147483647	
add = 1	y = -2147483648 (WRAPPED!)
add = -1	y = 2147483646
add = 100	y = -2147483549 (WRAPPED!)
add = -100	y = 2147483547
add = 5	y = -2147483644 (WRAPPED!)
add = -5	y = 2147483642
add = 1000000	y = -2146483649 (WRAPPED!)
add = -1000000	y = 2146483647
add = 2	y = -2147483647 (WRAPPED!)
add = -2	y = 2147483645

5/10 additions hit undefined behavior (overflow) — every one of them "looked successful," none crashed. UBSan catches these at run time.

Animation — Edge Case: the Secure Version Rejects the Same Input

`x = INT_MAX`; for each `add`, is it a bare `x + add`, or `checked_add(x, add, &y)`?

`INT_MAX = 2147483647`

`add = 1`

REJECTED — `y` was never written

`add = -1`

accepted: `y = 2147483646`

`add = 100`

REJECTED — `y` was never written

`add = -100`

accepted: `y = 2147483547`

`add = 5`

REJECTED — `y` was never written

`add = -5`

accepted: `y = 2147483642`

`add = 1000000`

REJECTED — `y` was never written

`add = -1000000`

accepted: `y = 2146483647`

`add = 2`

REJECTED — `y` was never written

`add = -2`

accepted: `y = 2147483645`

5/10 additions were rejected; the rest were accepted with the correct result. No overflow slipped through SILENTLY.



Error Handling and Signals

Signal Handler — Diagram

Sinyal işleyicide ne yapılır?

sinyal, kesilen kodun ortasında gelebilir

YANLIŞ

printf, malloc, free
kilit alan her şey

yeniden girişte iç yapı bozulur
→ kilitlenme / çökme

DOĞRU

yalnız bayrak kur:
`volatile sig_atomic_t`

asıl işi ana döngü yapar
(`write()` gibi güvenli çağrılar)

Async-signal-safe olmayan fonksiyon çağırmak tanımsız davranıştır.

Unchecked Return Value

Call	If Not Checked
<code>malloc</code>	Writing to NULL
<code>setresuid</code>	Stays privileged
<code>RAND_bytes</code>	The "random" key is zero
<code>EVP_DecryptFinal_ex</code>	Tampered data is assumed verified

Rules for Error Handling

1. Check every return value (ERR33-C); `[[nodiscard]]`.
2. On error, return to a **safe state** (`goto cleanup`) — **fail-closed**.
3. The message must not give a hint: "no such user" $\neq$ "wrong password."

From the Field: Opaque Return Values

- Native functions do not return a detailed **error code**.
- Only success/failure, and even that is **opaque** (Week 9).
- A detailed code = a map for the attacker of "which check you tripped."
- A separate, secure channel for diagnosis → recorded as a **trade-off**.

Signal Handler · Rule

```
static volatile sig_atomic_t stop = 0;
static void handler(int s){ (void)s; stop = 1; } /* ONLY a flag */
```

- No `printf`, `malloc`, `free` in the handler (SIG30-C).
- Do not call functions that are not async-signal-safe.
- `sigaction()` instead of `signal()`.

Real Incident: regreSSHion

- 2024 CVE-2024-6387: the timeout **signal handler** called an unsafe logging function.
- Result: remote code execution risk.
- Lesson: only set a flag inside a handler.

Section 2–3 — Quick Check

1. Why is `%n` the most dangerous?
2. The three traces in a UAF report? Which one gets the fix?
3. Why can `if (x+100 < x)` be deleted?
4. Why is there no `printf` in a signal handler?

Section 2–3 — Answers

1. `%n` **WRITES** the number of bytes printed so far **to memory** → an attacker can write to an address of their choosing and hijack control flow.
2. **allocated-by · freed-by · use-here**. The fix is usually **between freed↔use**: set the pointer to `NULL` after `free`, or fix the lifetime.
3. Signed overflow is **undefined behaviour**; the compiler assumes "overflow never happens" → the condition is always `false` → it **deletes** it. Do the check with `__builtin_add_overflow`.
4. `printf` is **not async-signal-safe**; if the handler re-enters it, locks/state get corrupted → deadlock/crash. Only safe calls like `write` are allowed.

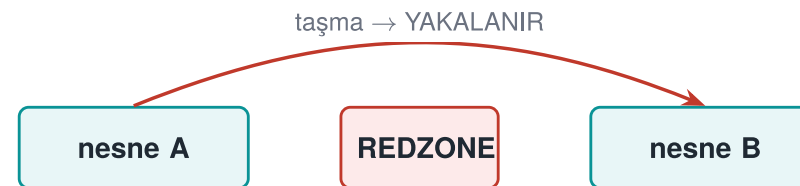


4. Static Analysis, Sanitizers, Fuzzing

ASan Redzone — Diagram

AddressSanitizer neyi yakalar, neyi kaçıır?

nesneler arasına “zehirli bölge” (redzone) koyar



alan1 | alan2
aynı struct içinde

aralarında redzone YOK
→ KAÇIRILIR

ASan ayrıca hiç çalışmayan kod yolunu da göremez — fuzzing bu yüzden gerekir.

Static ↔ Dynamic — Diagram

Statik analiz ↔ dinamik analiz

biri diğerinin yerine geçmez — ikisi birden kullanılır

STATİK (çalıştırmadan)

tüm kod yollarına bakar
hiç çalışmayan yolu da görür

yanlış pozitif ÇOK
bağlamı bilmez

DİNAMİK (çalıştırarak)

yalnız yürütülen yol
gerçek hatayı KESİN görür

yanlış pozitif AZ
kapsam girdiye bağlı

Fuzzing üçüncü ayakdır: dinamik analizin göreceği YENİ yolları üretir.

What Is Static Analysis?

Examining code **without running** it, looking for bugs.

Advantage: it also sees paths that never execute.

Disadvantage: it produces **false alarms**.

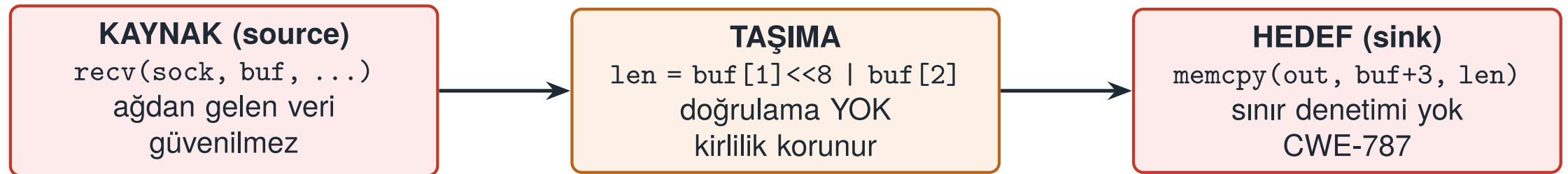
Static Analysis · Layer by Layer

Level	Tool	What It Finds
Warnings	<code>-Wall -Wextra -Wconversion</code>	Conversion, format
Analyser	<code>-fanalyzer</code> , <code>clang --analyze</code> , <code>/analyze</code>	NULL, leaks
Rule	<code>clang-tidy</code> <code>cert-*</code> , <code>cppcheck</code>	CERT violation
Semantic	CodeQL, Semgrep	Data flow
Commercial	Coverity, Klocwork	Deep analysis

Tainted Data: Source → Sink

Kirli veri takibi: kaynaktan hedefe

saldırganın yazdığı bir değer, denetlenmeden tehlikeli bir çağrıya ulaşıyor



Bulgu kuralı: kaynak ile hedef arasında doğrulama yoksa, o yol bir zafiyettir.

The tool sees **unvalidated** data flowing to a dangerous place.

Living with False Alarms

1. Start with **high-confidence** rules.
2. Every suppression has a **reason**: `// NOLINT(cert-err33-c): neden .`
3. **Zero warnings** on new code (enforced by CI).

The evaluator asks for the developer's own scan report.

What Is a Sanitizer?

A tool that catches memory/UB bugs while the program runs.

Almost no false alarms; but it only finds bugs on the path that **actually executes**.

→ combine with tests + fuzzing.

Brief History · AddressSanitizer

- **2012** — Google released **AddressSanitizer** (ASan): compile-time instrumentation + shadow memory catches overflow/UAF **immediately**.
- Before this, this class of bug usually crashed **silently** or went unnoticed entirely.
- ASan + fuzzing together catch input that doesn't crash the program but does corrupt memory (this week's Sections 4/11).

The Sanitizer Family

Sanitizer	Finds	Flag
ASan	Overflow, UAF, double free	<code>-fsanitize=address</code>
LSan	Leaks	with ASan
UBSan	Overflow, shift, NULL	<code>-fsanitize=undefined</code>
MSan	Uninitialized reads	<code>-fsanitize=memory</code>
TSan	Data race	<code>-fsanitize=thread</code>

How Does ASan Work? · Shadow Memory

- One **shadow byte** per 8 bytes: how much of it is valid?
- A **poisoned region** (redzone) before/after arrays.
- Freed blocks wait in **quarantine**.
- A shadow check before every access.

ASan's Blind Spot

```
struct { char name[8]; long role; } k;  
strcpy(k.name, long_input); /* name → role overflows */
```

A field-to-field overflow **inside** the same structure → no poisoned region in between → ASan **cannot see it** (Week 1 Demo 3).

Sanitizers Do Not Ship in the Release Build

```
gcc -g -O1 -fno-omit-frame-pointer \  
-fsanitize=address,undefined p.c -o p_test
```

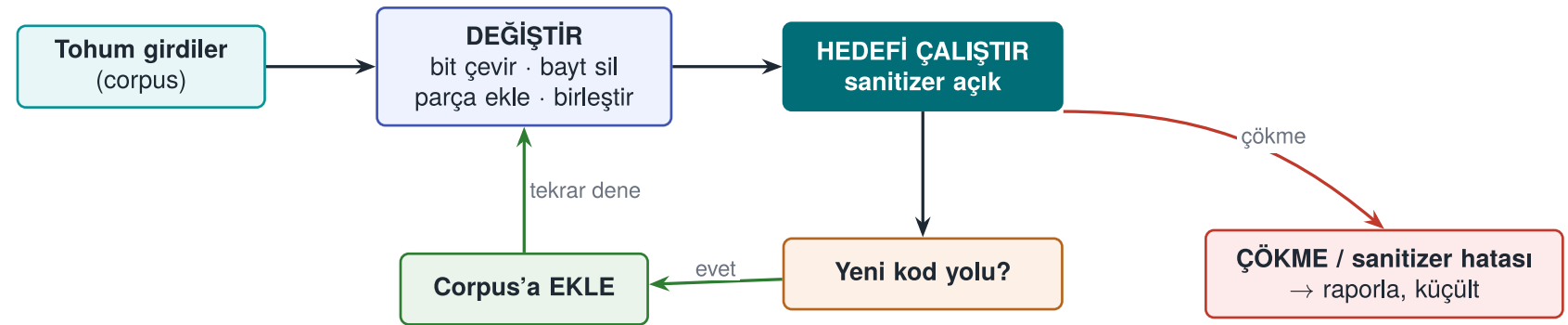
- Slows things down ($\sim 2\times$), uses memory.
- Only in **test/CI**; disabled in the release build.

What Is Fuzzing? · Coverage-Guided

Fuzzing: feeding a program a large number of automatically generated, mostly malformed inputs to look for crashes.

Kapsam güdümlü fuzzing döngüsü

rastgele değil: yeni kod yolu bulan girdiyi saklar



İyi fuzz hedefi: hızlı · deterministik · kendine yeterli · dar girdi yüzeyi.

An input that opens a new code path is **saved** and worked on further.

What Does Fuzzing Find?

- It finds magic bytes and the right length field **on its own**.
- Inputs a human would never think of.
- Tools: libFuzzer, AFL++, honggfuzz, OSS-Fuzz (tens of thousands of bugs).

Brief History · Fuzzing

- **1990** — Barton Miller's "Fuzz" study crashed 25-33% of Unix utilities with random input; the academic origin of fuzzing.
- **2013** — **AFL** (american fuzzy lop) made coverage feedback practical and mainstream.
- Then came **libFuzzer** (built into the compiler, fast) and OSS-Fuzz (a service that continuously fuzzes major open-source projects).

Writing a Fuzz Target

```
int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {  
    parse_document((const unsigned char *)data, size);  
    return 0;  
}
```

```
clang -g -O1 -fsanitize=fuzzer,address fuzz.c parser.c -o f  
./f corpus/ -max_total_time=10
```

A Good Fuzz Target

Four properties:

- **Fast** (no network/disk)
- **Deterministic** (same input → same result)
- **Stateless** (not affected by the previous call)
- **Single job** (tests one function)

Demo 4 — Finding Bugs with Fuzzing

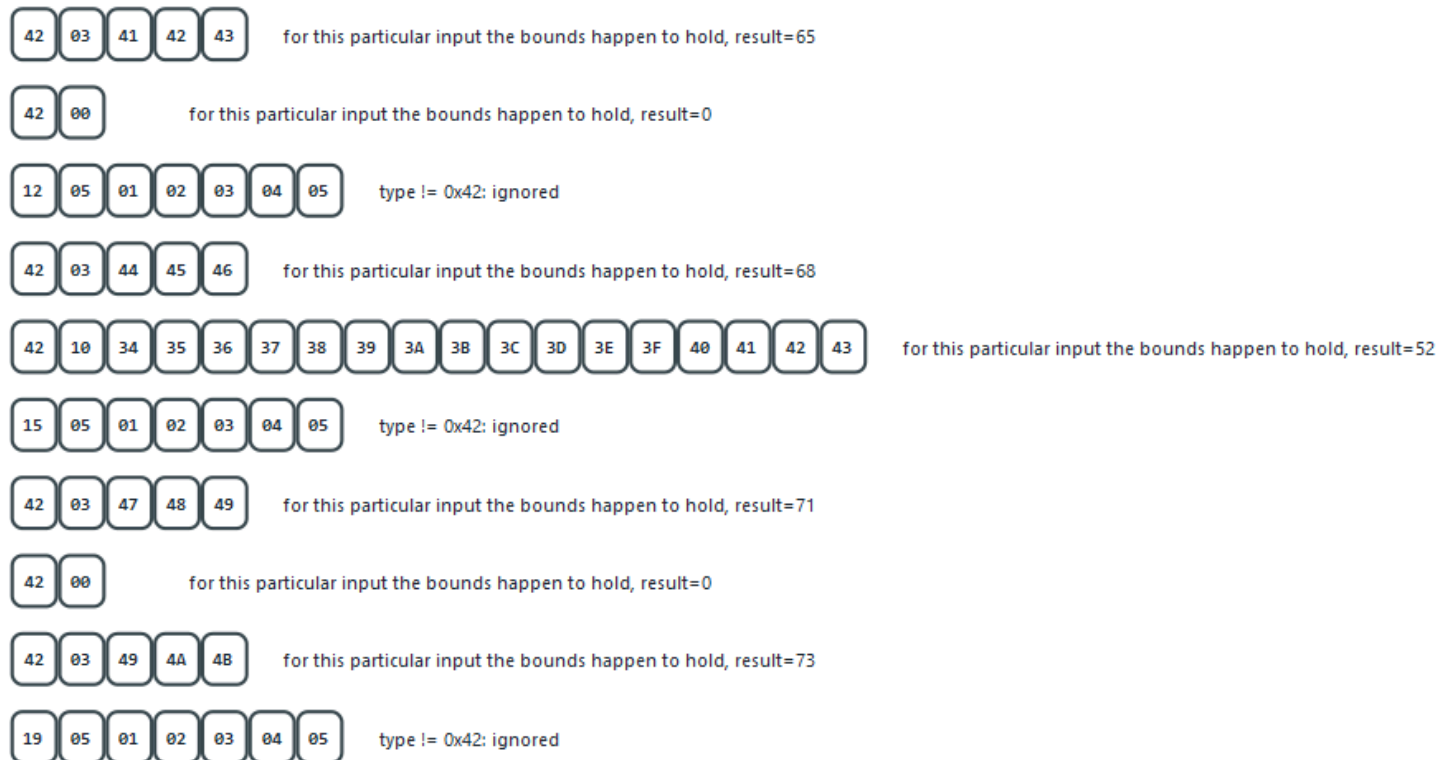
code/week-04/04-fuzzing · CWE-125/787

Step	What Happens?
1	<code>seeds/normal.bin</code> is fine
2	<code>libFuzzer</code> ≤ 10 s $\rightarrow$ a crashing input
3	Replay with ASan $\rightarrow$ full report
4	Fixed version $\rightarrow$ no crash
5	<code>crash.bin</code> is safely rejected

"This function was fuzzed for this long" = strong evidence (S16).

Animation — an allow-list parser

parser (buggy): no checks — the length and data are read directly



0/10 records caused an out-of-bounds access (ASan flags these as "heap-buffer-overflow"). Treating the rest as safe "because we got lucky" would be wrong.

Animation — Edge Case: Every Record Is Malformed

parser (buggy): no checks — the length and data are read directly

(empty input) $n < 1$: safely returns 0

- 42 $n=1$: `data[1]` is READ but does not exist — OUT-OF-BOUNDS READ (undefined behavior)
- 42 13 $\text{length}=19$ was never checked: the loop reads up to `data[2..20]` — OUT OF BOUNDS (past the input and/or the 16-byte destination)
- 42 0A 01 02 03 $\text{length}=10$ was never checked: the loop reads up to `data[2..11]` — OUT OF BOUNDS (past the input and/or the 16-byte destination)

(empty input) $n < 1$: safely returns 0

- 42 $n=1$: `data[1]` is READ but does not exist — OUT-OF-BOUNDS READ (undefined behavior)
- 42 11 $\text{length}=17$ was never checked: the loop reads up to `data[2..18]` — OUT OF BOUNDS (past the input and/or the 16-byte destination)
- 42 0A 01 02 03 $\text{length}=10$ was never checked: the loop reads up to `data[2..11]` — OUT OF BOUNDS (past the input and/or the 16-byte destination)

(empty input) $n < 1$: safely returns 0

- 42 $n=1$: `data[1]` is READ but does not exist — OUT-OF-BOUNDS READ (undefined behavior)

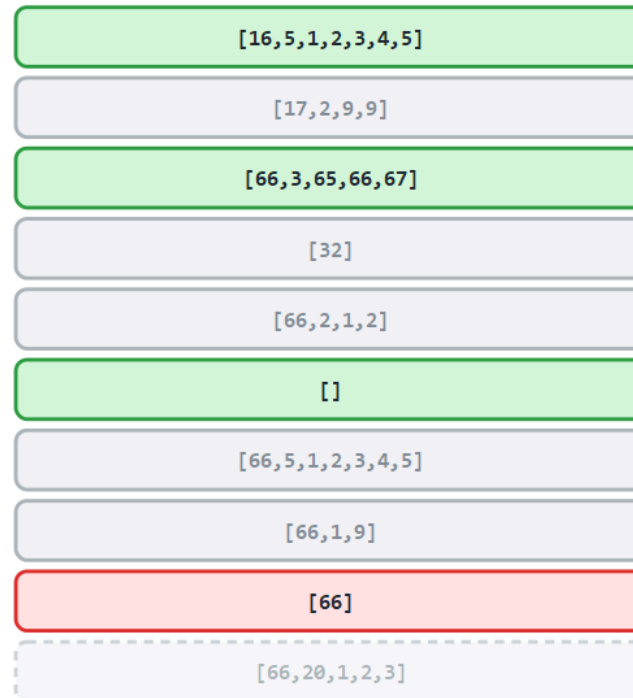
7/10 records caused an out-of-bounds access (ASan flags these as "heap-buffer-overflow"). Treating the rest as safe "because we got lucky" would be wrong.

Animation — coverage-guided fuzzing

Coverage-guided fuzzing: an input that opens NEW coverage is KEPT, a repeat is SKIPPED

corpus (kept inputs):

[#1, #3, #6, #9]



no length byte ($n < 2$) — CRASH

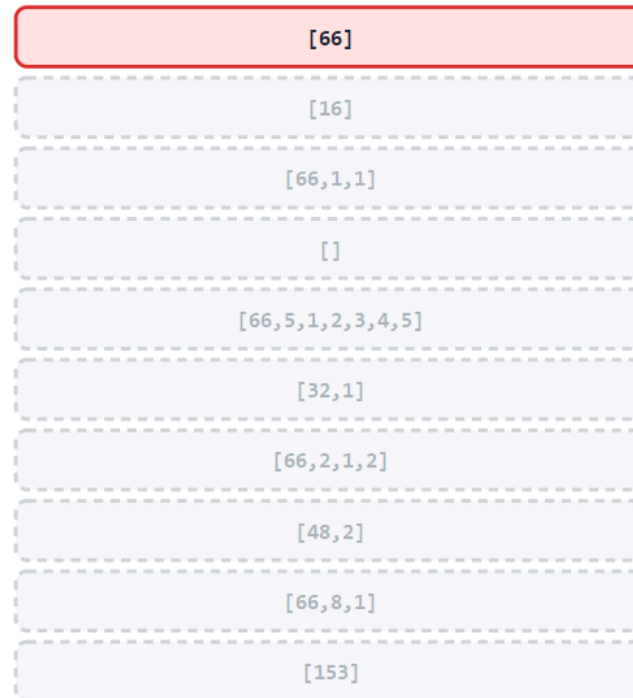
4 regions were discovered; a crash was found at candidate 9 (1 candidate(s) were never tried — the session stopped).

Animation — Edge Case: the Very First Candidate Crashes

Coverage-guided fuzzing: an input that opens NEW coverage is KEPT, a repeat is SKIPPED

corpus (kept inputs):

[#1]



no length byte ($n < 2$) — CRASH

1 regions were discovered; a crash was found at candidate 1 (9 candidate(s) were never tried — the session stopped).

Section 4 — Quick Check

1. The difference between static and dynamic analysis?
2. Which overflow can ASan not see?
3. The four properties of a good fuzz target?

Section 4 — Answers

1. **Static:** examines all paths without running the code (can have false positives). **Dynamic:** while running, observes **real** bugs only on the path that actually executes.
2. ASan sees the places where it placed a **redzone**; it cannot see a **field-to-field** overflow inside an object, or paths that **never execute**.
3. **Fast, deterministic, self-sufficient** (not dependent on external state), a target that handles a **narrow input surface** and fails cleanly on a crash.

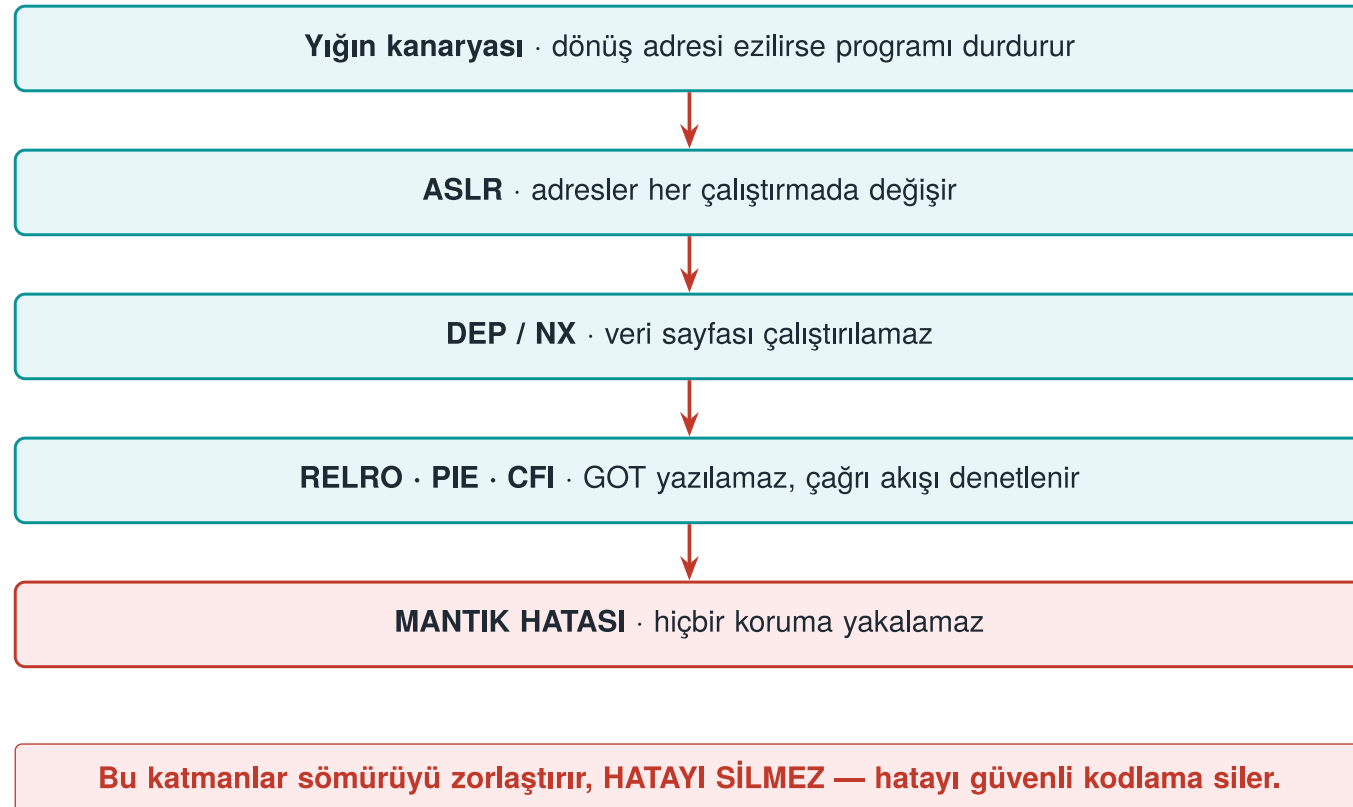


5. Compiler and OS Protections

Compiler/OS Protections — Diagram

Derleyici ve işletim sistemi korumaları

saldırgan yukarıdan aşağı her katmanı ayrı ayrı aşmak zorunda



Layer 3 · the Idea

Say a bug slipped through.

- There are automatic protections that make exploitation **expensive**.
- Most are turned on with a **compile flag**.
- Not free, but cheap.

Stack Canary

- A **sentinel value** is placed just before the return address.
- If an overflow corrupts this value, it is noticed before the return and the program **halts**.
- `-fstack-protector-strong` / MSVC `/GS` .

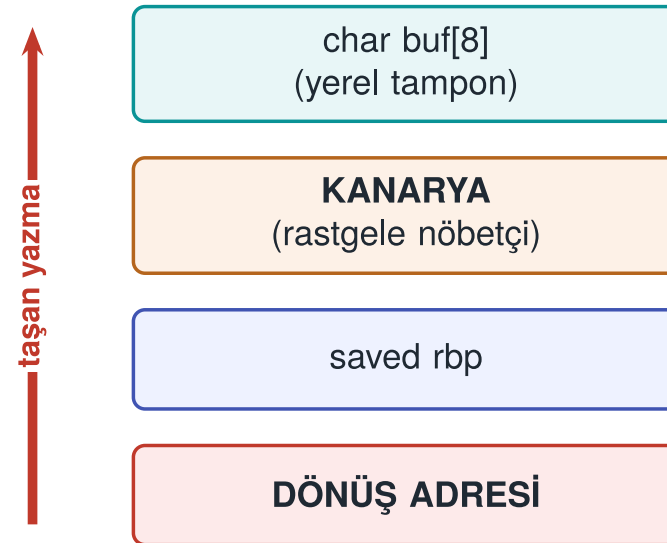
Brief History · Compiler/OS Protections

- **1998** — **StackGuard**: the first to add the stack-canary idea to a compiler automatically.
- **2001** — the **PaX** project brought **ASLR** (address space layout randomization) to the Linux kernel.
- **2003–2004** — **W^X / NX / DEP**: a memory page can no longer be both writable and executable at once.
- Together, these three form the foundation of today's compiler/OS protections.

Stack Overflow and Canary — Diagram

Yığın taşması ve kanarya

taşma tampondan dönüş adresine doğru yazar



Dönüş adresine ulaşmak için önce KANARYA ezilmek zorunda.

Dönüşte kanarya bozuksa program DURUR.

Durdurmaz: hedefli yazma · öbek (heap) taşması · bilgi sızıntısı.

FORTIFY_SOURCE

- Catches **library calls** that write more than a buffer's known size.
- `-D_FORTIFY_SOURCE=2` (+ at least `-O1`).
- Does not protect your **own loops**.

PIE + ASLR

- **ASLR:** randomizes addresses.
- **PIE:** makes the program itself load at a random address too.
- `-fPIE -pie / /DYNAMICBASE /HIGHENTROPYVA`.

DEP/NX and RELRO

- **DEP/NX:** makes the data region non-executable (default).
- **RELRO:** makes tables like the GOT read-only → `-Wl,-z,relro,-z,now`.

CFI / CFG and the Shadow Stack

- **CFI/CFG:** checks that indirect calls only go to **valid** targets.
- `-fcf-protection`, `-fsanitize=cfi` / `/guard:cf`.
- **Shadow stack:** a second copy of the return addresses (CET).

Protections · One Table

Protection	GCC/Clang	MSVC
Canary	<code>-fstack-protector-strong</code>	<code>/GS</code>
FORTIFY	<code>-D_FORTIFY_SOURCE=2</code>	Secure CRT
PIE/ASLR	<code>-fPIE -pie</code>	<code>/DYNAMICBASE</code>
NX	default	<code>/NXCOMPAT</code>
RELRO	<code>-Wl,-z,relro,-z,now</code>	—
CFI	<code>-fcf-protection</code>	<code>/guard:cf</code>

Every Protection Has Limits

Protection	Stops	Does Not Stop
Canary	Overflow reaching the return address	Heap overflow, a leaked canary
FORTIFY	Library-call overflow	Your own loops
ASLR	Assuming a fixed address	Address leak
NX	Code in the data region	Code reuse (ROP)

The Main Rule

Every protection makes a **technique** harder; it does not remove a **class of bug**.

None of them is enough for a logic error → secure coding comes first.

Demo 5 — Turning Protections On and Off

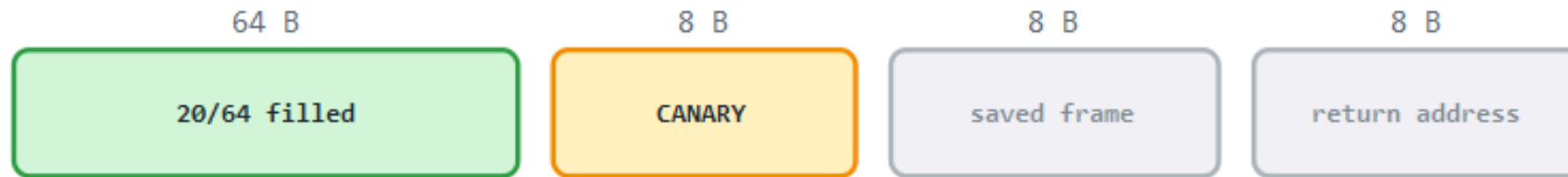
code/week-04/05-compiler-protections · CWE-121

- The same overflow: `overflow_weak` (off) vs `overflow_hardened` (on).
- Hardened: `*** stack smashing detected ***` / Windows `0xC0000409`.
- Weak: silent corruption or a crash.

```
readelf -h p | grep Type          # DYN = PIE
readelf -d p | grep BIND_NOW      # full RELRO
readelf -s p | grep __stack_chk   # canary
```

Animation — the stack canary

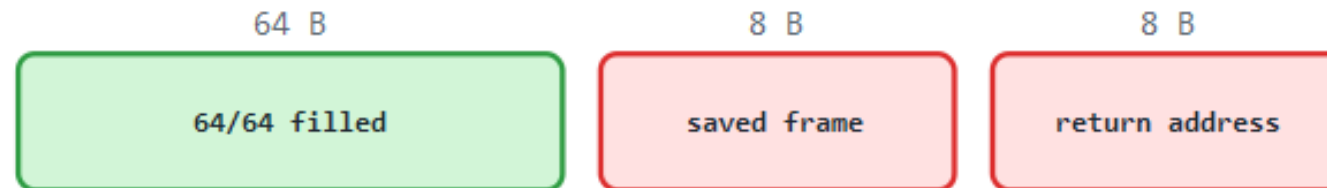
strcpy(buffer, input) — once the input crosses the 64-byte bound, it writes into the neighbouring regions IN ORDER



20 characters fit inside buffer exactly (no overflow). The function returns normally; the canary (if present) is never touched.

Animation — Edge Case: Same Overflow, WEAK Build Has No Canary

strcpy(buffer, input) — once the input crosses the 64-byte bound, it writes into the neighbouring regions IN ORDER



This build has NO canary to check: the function returns "normally," but the saved frame pointer and/or return address are now the attacker's data — the NEXT step (the function returning) may crash, or worse.

Recommended Release Flags

GCC/Clang:

```
-O2 -D_FORTIFY_SOURCE=2 -fstack-protector-strong -fPIE -pie -Wl,-z,relro,-z,now -fcf-protection -Wall -Wextra -Wformat=2 -Werror=format-security
```

MSVC:

```
/O2 /GS /sdl /guard:cf /DYNAMICBASE /HIGHENTROPYVA /NXCOMPAT /CETCOMPAT /W4
```

Protection Table = Evidence

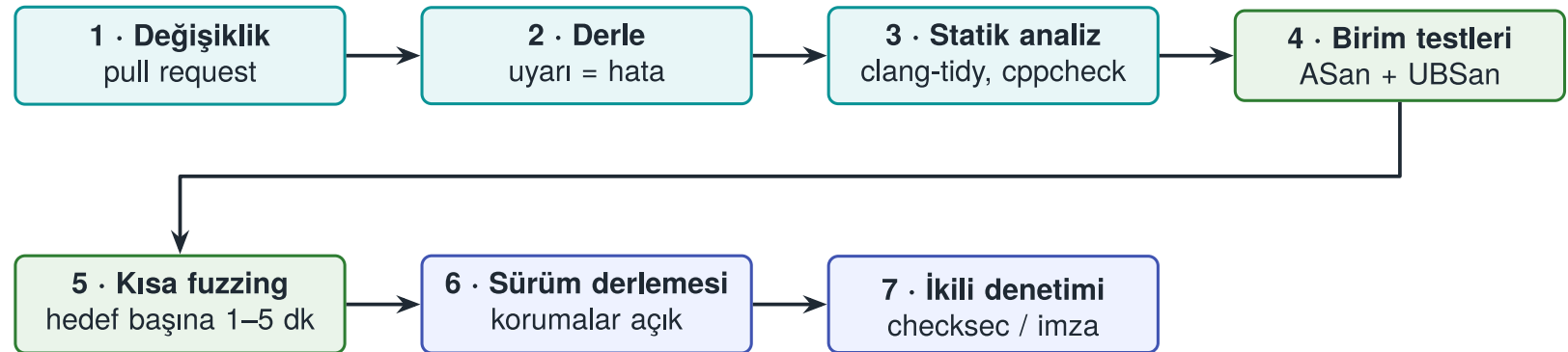
- The evaluator extracts a **protection table** for every binary and `.so`.
- A disabled protection is a **finding**; its justification must be in the trade-off record.
- Scheme: the application can also **verify itself** that the protections are enabled.

Secure Build Pipeline (CI)

CI (Continuous Integration): a system that runs an automatic build + tests on every code change.

Güvenli derleme hattı (CI)

her adım geçmeden bir sonrakine geçilmez



Ucuzdan pahalıya: erken adımlar kolay hataları eler, pahalı adım yalnız kalanlara harcanır.

Every failing step **stops** the merge.

CI · Failure Criteria

Stage	Failure
Build	A single warning
Sanitizer-enabled test	Any report
Fuzzing	A new crash (old corpus = regression)
Binary audit	Log strings, symbols, a disabled protection

The Course's CMake Modes

Mode	Flags
korumasiz	-00
optimize	-02
asan	-01 -fsanitize=address
denetimli	-02 -D_FORTIFY_SOURCE=2
guvenli	-02 + fixed source

Most of the protection flags: Demo 5's `overflow_hardened`.

The Build Environment Is a Target Too

- Access logging; pinning dependencies (Week 5's SBOM).
- Signed releases; **reproducible** builds.
- If the build machine is compromised, the entire product is affected.

Section 5 — Quick Check

1. What does the canary stop, and what does it not stop?
2. Why is ASLR weakened by an information leak?
3. Why is "no protection is enough for a logic error"?

Section 5 — Answers

1. **Stops:** a **sequential** stack overflow that overwrites the return address (the canary gets corrupted). **Does not stop:** a targeted write, a heap overflow, an information leak.
2. A single **leaked address** reveals the module's base address (the offset is fixed) → the entire randomisation **collapses**.
3. Protections catch **memory** violations; incorrect authorization/business logic is a **valid** memory access → no protection is triggered. Correct design + review is required.



6. Introduction to Code Obfuscation

Layer 4 · Why?

- White-box attacker: reads strings, finds functions by name, bypasses checks.
- Obfuscation: same behaviour, **harder to understand**.

Two Attacker Models — Diagram

Uzak saldırgan mı, cihazın sahibi mi?

aynı programa iki farklı tehdit modeli

UZAK SALDIRGAN

girdi gönderir
belleği göremez
bellek güvenliği + doğrulama yeter
1–5. haftalar

CİHAZIN SAHİBİ (MATE)

ikiliyi elinde tutar
hata ayıklayıcı bağlar
gizleme + RASP gerekir
6., 9., 14. haftalar

Gizlemeye buradan geçiyoruz: bellek güvenliği ikinci saldırgana karşı yetmez.

The Purpose of Obfuscation

Not making it impossible, but making it **expensive** (Cookbook 12.1):

- cost > the value being protected
- make automation harder (diversification, Week 14)
- buy **time** for the other layers

A few simple techniques + data hiding, rather than one advanced technique.

Brief History · a Taxonomy of Obfuscation

- **1997** — Collberg, Thomborson, and Low published the work that organized obfuscation techniques into a **systematic taxonomy**: layout, data, control, and preventive transformations.
- What we saw this week — string/symbol hiding = a **data** transformation; control-flow flattening and the opaque predicate = a **control** transformation.
- The taxonomy is still the shared vocabulary of academic and industrial obfuscation literature.

Obfuscation Map

Family	What It Hides	Week
Layout	Names, structure	4
Data	Constants, strings, variables	4, 9
Control flow	Algorithm structure	4, 9
Preventive	Analysis tool	9
Virtualisation	Machine code	9, 14

From the Field: 18 Measures on the Native Side

Symbol visibility off · name mangling · constant arithmetic hiding · string encryption (decrypt on use, **wipe immediately**) · opaque booleans · your own library functions · fake operation/dead branch · flattening + randomised exit · **no** logging in the release build.

➔ The strength comes from **combining them**, and from RASP.

Three Cheap Measures

Measure	How?
Hide symbols	<code>static</code> , <code>-fvisibility=hidden</code> , <code>strip -s</code>
Remove logging	Empty the macro at compile time
Hide strings	Encrypt at compile time, decrypt on use, wipe immediately

Logging Macro

```
#ifndef LOG_ENABLED
#  define LOG(...) fprintf(stderr, __VA_ARGS__)
#else
#  define LOG(...) ((void)0) /* neither the string nor the call end up in the binary */
#endif
```

`LOG_ENABLED` is not defined in the release build → the logging is **never compiled in**.

a Runtime Flag Is Not Enough

```
if (debug_flag) printf("..."); /* BAD */
```

- The string **remains** in the binary.
- The attacker can flip the flag and turn logging back on.
- The right way: remove it **at compile time**.

String Hiding ≠ Key Storage

- The decrypted string sits **in memory, in the clear**, while it's in use.
- The decryption key is **inside** the program.
- What it stops: static scans like `strings`.
- Against someone inspecting it while running → RASP (6). A real key → whitebox (11).

Demo 6 — Symbol and String Leakage

code/week-04/06-symbol-strings · CWE-200/215

Step	What Happens?
1	<code>secret_exposed</code> and <code>secret_hidden</code> give the same result
2 <code>strings</code>	The license string and <code>[LOG]</code> are visible in the open one; not in the closed one
3 <code>nm</code>	<code>license_verify</code> is visible in the open one; no symbol in the closed one
4	Windows: names are in the PDB; the closed build produces no PDB

Animation — string + symbol hiding

EXPOSED build: the string sits in the binary as PLAIN text

XOR key = 0x5A

C	E	N	4	2	9	-	L	I	C	E	N	S	E	-	2	0	2	6
43	45	4E	34	32	39	2D	4C	49	43	45	4E	53	45	2D	32	30	32	36

symbols visible with nm:

`license_verify`

`secret_text`

Summary: 19/19 bytes are still in the printable range — the whole string appears EXACTLY as-is in strings output.

Animation — Edge Case: a Bad Key Choice (0x00)

HIDDEN build: every byte is XOR'd with a key at compile time

XOR key = 0x00

W	E	A	K	-	X	O	R	-	K	E	Y	-	D	E	M	O	0	1
57	45	41	4B	2D	58	4F	52	2D	4B	45	59	2D	44	45	4D	4F	30	31

symbols visible with nm:

(stripped: no symbol)

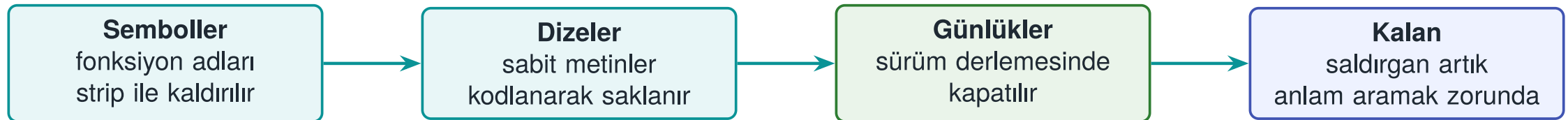
(stripped: no symbol)

Summary: 19/19 bytes are still in the printable range — but the WHOLE run landing there is rare; strings will NOT see this as one readable word.

Symbol and String Hiding — Diagram

Sembol, dize ve günlük gizleme: ilk savunma hattı

tersine mühendisliğin en ucuz adımını pahalılaştırmak



En ucuz koruma budur ve çoğu projede eksiktir: `strings` çıktınızı bir kez okuyun.

Control-Flow Flattening

```
int state = 1;
for (;;) switch (state) {
    case 1: state = 2; break;
    case 2: state = (condition ? 3 : 4); break;
    case 3: state = 5; break;
    case 4: return FAILURE;
    case 5: return SUCCESS;
}
```

The blocks' natural adjacency is lost; the order lives only in the **state variable**.

Flattening: Before/After — Diagram

Kontrol akışı düzleştirme ve opak değerler

aynı davranış, okunamaz grafik

ÖNCE

if / else / while
grafik okunabilir
algoritmanın iskeleti görünür
birkaç temel blok

SONRA

tek switch + durum değişkeni
sıra değişkende saklı
opak yüklemeler sahte dal ekler
çok sayıda blok

Maliyeti vardır: boyut ve hız artar. 9. ve 14. haftada ölçmeyi öğreneceğiz.

What Strengthens Flattening

- **Hide the state values** (arithmetic transformation)
- **Fake/dead blocks** (which one is real?)
- **Randomised exit** (where the error was caught becomes unreadable)
- **Opaque values** (a single byte cannot flip it)

The template alone is solved quickly on its own → Week 9 depth.

Demo 7 — Flattening

code/week-04/07-flow-flattening

Step	What Happens?
1	<code>pin.c</code> and <code>pin_flattened.c</code> give the same result (the PIN is synthetic)
2	<code>objdump</code> : the flattened version has far more branches
3	Timing: the cost of flattening

→ Automated with Tigress's `Flatten` in Week 14; measuring effectiveness in Week 9.

Animation — the dispatcher state machine

Natural if-chain: returns immediately at the first failed check

`strlen == 4?`

`guess[0]=='4'?`

`guess[1]=='2'?`

`guess[2]=='9'?`

`guess[3]=='1'?`

2/10 guesses were correct. Both versions give the EXACT SAME result — only the control flow differs.

Animation — Edge Case: Every Guess Has the Wrong Length



0/10 guesses were correct. Both versions give the EXACT SAME result — only the control flow differs.

Animation — the opaque predicate

	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10
$x =$	0	1	2	3	4	5	6	7	8	9
$x * x =$	0	1	4	9	16	25	36	49	64	81
$(x * x) \% 4 =$	0	1	0	1	0	1	0	1	0	1

``pin_verify_obfuscated("4291", x)`` — does the result **DEPEND** on x ?

$x=0 \rightarrow 1$

$x=9 \rightarrow 1$

10/10 values confirm $(x * x) \% 4 \neq 2$ — 2 never appeared. The decoy branch (decoy_branch) is a dead path a reverse engineer must read, but it never runs.

Animation — Edge Case: Values Near the INT Bounds

	x1	x2	x3	x4	x5	x6	x7	x8	x9	x10
a	2147483647	-2147483648	0x100	-0x100	0	1	-1	2	-2	0x100
a*a	0x11888888111122220000	0x11888888111122220000	2147483648	2147483648	0	1	1	0	0	0x11888888111122220000
(a*a)%4	1	0	0	0	0	1	1	0	0	0

'pin_verify_obfuscated('4291', x)' — does the result DEPEND on x?

a=2147483647 -> 1

a=-2147483648 -> 1

10/10 values confirm $(x*x)\%4 \neq 2$ — 2 never appeared. The decoy branch (decoy_branch) is a dead path a reverse engineer must read, but it never runs.

Concept: Three Techniques Leading into Week 9

- **Function name hiding:** exported names are meaningless; readability handled via a macro.
- **Allocation-size hiding:** blur the "32 bytes = a key" tell.
- **Dynamic decryption:** sensitive code sits encrypted in the binary; conflicts with DEP/NX, used selectively.

Section 6 — Quick Check

1. Does obfuscation fix the bug?
2. Why isn't silencing logging with a runtime flag enough?
3. Three techniques that strengthen flattening?

Section 6 — Answers

1. **No.** Obfuscation only makes reverse engineering harder; the bug **stays where it is**. **Fix first, then obfuscate.**
2. The string/code **remains** in the binary; the attacker can flip the flag or read the strings. Remove logging in the release build **at compile time** (macro/dead-code elimination).
3. **Opaque predicates + fake blocks/dead branches + state-variable encoding & randomised exit** (also name/string hiding).



End to End: Closing One Overflow in Three Layers

Buggy Code

```
void greet(const char *name) {  
    char buffer[16];  
    strcpy(buffer, name);      /* no bound */  
    printf("Hello %s\n", buffer);  
}
```

What happens if `name` is longer than 16 bytes?

What Happens? · Step by Step

- `buffer` is 16 bytes on the stack.
- `strcpy` copies `name` all the way to its end, paying no attention to bounds.
- A `name` longer than 16 → neighbouring memory (including the return address) is corrupted.

Attack · Outcome

- Short overflow: a neighbouring variable gets corrupted (a logic bug).
- Long overflow: the **return address** gets corrupted → crash or control hijacking.
- Without protection: a serious vulnerability (CWE-121).

Fix 1 · Bounded Copying

```
int n = snprintf(buffer, sizeof buffer, "%s", name);
if (n < 0 || (size_t)n >= sizeof buffer) {
    /* truncated: reject or flag it */
}
```

`snprintf` knows the size and **reports truncation**.

Fix 2 · Layers

- **Coding:** `snprintf` (STR31-C).
- **Compiler:** `-fstack-protector-strong` (canary).
- **FORTIFY:** `-D_FORTIFY_SOURCE=2` checks the library call.
- **Testing:** ASan catches the overflow with a long input.

Lesson: One Line, Many Layers

- One bug: `strcpy`.
- Defence: the right function + compiler protection + sanitizer testing.
- But the **real** fix is in the first line: bounded copying.

Classic Mistakes — Summary

Mistake	Section	Rule
Ignoring warnings	1	Turn warnings into errors with <code>-Werror</code> ; keep the CERT scan clean
"Cleaning up" input and using it anyway	1	Reject dangerous input; allow-list + rejection is safest
Not checking the return value	2–3	Check every return (ERR33-C); <code>fopen</code> / <code>malloc</code> can return NULL
Hiding UB with <code>-fwrapv</code>	2–3	Making the overflow "defined" hides the logic bug; diagnose with UBSan
Shipping the sanitizer in the release build	4	Test/CI only; disabled in the release build
Covering up a bug with obfuscation	6	Obfuscation doesn't fix it, only hides it; fix first, then obfuscate

All six mistakes were covered earlier in this deck; here they are gathered in one glance.



7. Project and Closing

Project · S9 "Code Hardening" — 1

- [] Build with the recommended release flags; attach the protection table (`checksec` / `dumpbin`).
- [] Run a CERT scan with `clang-tidy` / `cppcheck` ; fix ≥ 5 findings (with rule ids).

Project · S9 — 2

- [] Run the tests with ASan + UBSan.
- [] At least one fuzz target, ≥ 10 min.
- [] No logging in the release build; no sensitive string in `strings` output.
- [] Every return value is checked (ERR33-C); it builds with `-Werror`.

Measure card: Description → Implementation → Verification → Residual risk.



Solved Self-Check

Question 1

Which of the three layers actually removes the bug?

Answer: Only **secure coding** (layer 2). Compiler/OS protections make exploitation harder; obfuscation slows down reading — neither one **removes** the bug.

Question 2

What is the mildest outcome of `printf(girdi)` ?

Answer: An **information leak** (reading the stack via `%x` / `%p`). The worst is writing to memory via `%n` .

Question 3

The three stack traces in a UAF report? Which one gets the fix?

Answer: Use, free, allocation. The fix is usually at the **free** point (an ownership decision).

Question 4

Why can `if (x + 100 < x)` be deleted?

Answer: Signed integer overflow is **undefined behaviour**; the compiler assumes "overflow never happens" and can optimise the check away, **deleting** it.

Question 5

Why is there no `printf` in a signal handler?

Answer: `printf` / `malloc` are **not async-signal-safe**; if the handler calls them while interrupted mid-state, corruption follows. Only a flag is set (SIG30-C).

Question 6

Which overflow can ASan not see?

Answer: A **field-to-field** overflow inside the same structure (there is no poisoned region between them).

Question 7

The four properties of a good fuzz target?

Answer: Fast, deterministic, stateless, single job.

Question 8

What does the canary stop, and what does it not stop?

Answer: Stops: a stack overflow that reaches the return address. **Does not stop:** heap overflow, corruption of earlier variables, a leaked canary.

Question 9

Why is ASLR weakened by an information leak?

Answer: If one address leaks, the attacker can **calculate** the other addresses relative to it; the advantage of randomisation is lost.

Question 10

Why isn't silencing logging with a runtime flag enough?

Answer: The string **remains** in the binary and the attacker can flip the flag to turn logging back on. It must be removed **at compile time**.

Question 11

Three techniques that strengthen flattening?

Answer: Hiding state values, fake/dead blocks, a randomised exit point (and opaque values).

Question 12

Why is string hiding not the same as key storage?

Answer: The decrypted string is in memory, in the clear, while it's in use, and the decryption key sits inside the program; it only stops **static** scanning.

Summary: This Week in One Sentence

First **bug-free code** (CERT + sanitizer + fuzzing), then **limit the damage** (compiler/OS protections), and finally **make it harder to read** (obfuscation) — and no protection substitutes for secure coding.

Secure code is not written once; it is **checked with tools on every build**.

Glossary

Term	Meaning
UB	Undefined behaviour
UAF	Use-after-free
CERT/CWE	Rulebook / weakness catalogue
Sanitizer	Runtime bug detector
Fuzzing	Finding crashes with random input
Canary/ASLR/NX	Compiler/OS protections

Next Week

Week 5 — Java and Interpreted Languages

This week we dealt with manually managed memory in C/C++ and compiler/OS protections; Week 5 shows how the JVM eliminates most of these bugs (overflow, UAF, UB), but opens a new bug class such as injection (SQL, command, path).

The SEI CERT Java rules will be compared with today's SEI CERT C/C++ rules; we will also cover obfuscation with ProGuard/R8, dependency security, and SBOM.

Preparation: JDK 17+, Maven · `code/week-05` .

First correct code, then protection, finally obfuscation. The order matters.